KAVACH: A Model-Agnostic, Intent-Aware Jailbreak Defence Layer for LLMs with Tactic–Harm Decoupling and Hindi–Hinglish Canonicalization

Vaibhavi Das* [co-authors / supervisor]
*[Department, Institution, City, India]
vaibhavidasv2005@gmail.com

***Abstract*—LLM safety mechanisms can be bypassed by *jailbreak* prompts that wrap a harmful request in role-play, hypothetical framing, instruction overrides or obfuscation. Such attacks are especially effective in regional and code-mixed languages such as Hindi and romanized Hinglish. Existing guard models depend on their training data and decide with a binary allow/refuse output, which also refuses many legitimate prompts that merely *look* like attacks. We present KAVACH, a plug-in safety layer that wraps any LLM through a Python API or an OpenAI-compatible proxy; its core needs no training data. KAVACH (i) canonicalizes inputs by transliterating Devanagari, mapping Hinglish spelling variants to phonetic keys and decoding obfuscation; (ii) scores *what* is asked (harm) separately from *how* it is asked (evasion tactic); (iii) sends uncertain prompts to an LLM judge driven by *Intent Decomposition Prompting*, which removes the framing and rates the bare core request; and (iv) chooses among six graded actions, from answering under a hardened prompt to redirecting to a safe rewrite or blocking, with an adaptive attack memory and conversation-level risk tracking. On a 4,648-prompt held-out test set drawn from seven public sources in seven Indian languages, decoupling cuts false refusals of benign in-the-wild prompts from 11.1% for a tactic-based detector to 0.2%, while the model-free stage routes 71.8% of harmful prompts to the judge. Removing transliteration drops Hindi routing recall from 64.3% to 0%. With Gemini 3.5 Flash-Lite as judge on a stratified 186-prompt sample, the IDP judge refuses 81.2% of harmful prompts and intervenes on 90.6% with no false refusals. A naive yes/no judge on the same model refuses about the same share overall (80.9%) but does worse on India-specific harms (73.8% vs. 88.1%), multi-turn jailbreaks (85.7% vs. 95.2%) and romanized Hindi (50.0% vs. 75.0%). Finally, we fine-tune multilingual transformers as the cascade's router. XLM-R raises routing recall from 71.8% to 95.0% while sending 58% fewer benign prompts to the judge, and the learned cascade refuses 74–80% of harmful prompts end to end, up from 62.4%.**

***Index Terms*—LLM safety, jailbreak defence, prompt engineering, guardrails, multilingual NLP, code-mixing, Hinglish**

I. Introduction

Large language models (LLMs) are now part of customer support, education, health and government services, where they must answer a wide range of queries safely. Safety alignment remains brittle. Jailbreak prompts exploit *competing objectives*, for example "stay in character", and *mismatched generalisation*, for example encodings or low-resource languages the safety training never covered [1]. In-the-wild jailbreak communities produce new templates continuously [2], and translating a harmful request into a low-resource language raises attack success sharply [3], [4]. For India this gap is acute. Users write Hindi in Devanagari, in romanized form, and in code-mixed Hinglish with no standard spelling, and phonetic misspellings of sensitive words in Hinglish have recently been shown to bypass alignment [5].

Current defences fall into two families. *Learned guard classifiers* such as Llama Guard [6] and the recent Indic-language IndicGuard [7] need labelled training data and react to attack patterns they have already seen. *Rule and perturbation defences*, such as perplexity filtering, paraphrasing [8], SmoothLLM [9] and programmable rails [10], are mostly English-centric. Almost all of them return a binary decision. A binary guard faces an unpleasant trade-off: catching jailbreaks by their form (role-play, "ignore previous instructions") refuses the many benign prompts that share that form, the *exaggerated safety* problem [11].

We argue that a jailbreak is best modelled as a *harmful goal wrapped in an evasion tactic*, and that a guard should reason about the two axes separately. We present KAVACH (Hindi: "armour"), a model-independent layer placed in front of any LLM. Our contributions are:

1) **Tactic–harm decoupling** with a six-action graded policy (ALLOW, GUARD, REDIRECT, SAFE-REFUSE, BLOCK, SUPPORT). It keeps the LLM useful for benign prompts that use jailbreak-like framing and still blocks real jailbreaks.
2) **Canonicalize-then-judge for Hindi/Hinglish**: Devanagari transliteration, Hinglish *phonetic keys* and de-obfuscation, so a single rule set covers three writing forms of the same language.
3) **Intent Decomposition Prompting (IDP)**, a structured judge prompt that translates the input, removes the framing, extracts the core request, rates harm separately from tactics, and proposes a safe rewrite. Nonce-delimited spotlighting [12] protects the judge itself from injection.
4) **Adaptation without training**: an attack memory of

judge-confirmed jailbreaks and a decayed conversation-risk score against multi-turn escalation [13].

5) **A learned router** for the cascade: fine-tuned multilingual transformers (XLM-R, MuRIL, DistilmBERT) that decide which prompts need the LLM judge. We compare them with a classical baseline over several seeds and test generalisation to a data source unseen in training.
6) An open-source, pip-installable implementation with a drop-in OpenAI-compatible proxy, and an evaluation on seven public sources including India-specific multi-turn jailbreaks in seven languages.

II. Related Work

Jailbreak attacks. Manual role-play and "DAN" templates [2], goal hijacking and prompt leaking [14], optimised adversarial suffixes [15], multi-turn escalation [13] and language-switching attacks [3], [4] all exploit the gap between what a model can do and how far its safety training generalises [1]. CMP-RT [5] shows that Hinglish code-mixing combined with phonetic misspelling of sensitive words is an effective red-teaming probe.

Defences. Input filters include perplexity checks and paraphrasing [8]; SmoothLLM [9] aggregates responses to perturbed copies of the prompt; NeMo Guardrails [10] provides programmable dialogue rails. Llama Guard [6] fine-tunes an LLM as an input/output safety classifier. IndicGuard [7] fine-tunes a guard model on ten Indic languages, and IndicSafe [16] benchmarks how safety judgements drift across 12 Indic languages. KAVACH is complementary to these. It needs no fine-tuning, separates tactic from harm instead of producing one label, and acts through graded responses rather than a refusal alone.

III. The Kavach Layer

A. Threat model

The adversary sends single- or multi-turn text to an LLM application and tries to obtain content that gives real-world uplift for serious harm. They may use any framing, any language or script, and obfuscation. The defender controls only a layer between the user and a black-box model. We also want to preserve usefulness: benign requests must not be refused merely because they resemble attacks.

B. Stage 1: Canonicalization

Each prompt x becomes a set of views. We apply NFKC normalisation and remove zero-width characters, and map Cyrillic/Greek homoglyphs to Latin. Devanagari is transliterated to Roman with an ITRANS-like mapping that includes Hindi word-final schwa deletion, and long vowels are additionally collapsed (*banaane→banane*) to meet colloquial romanization. Letter-spaced words, leetspeak inside words and embedded base64 payloads are decoded into extra views. Every transformation that changed the text is recorded as an *obfuscation signal*. For lexical matching of romanized Hindi we define a *phonetic key* $\kappa(w)$: aspirates are merged (*kh→k*, *sh→s*), $w \to v$, $z \to j$, $q \to k$, repeated letters are collapsed and non-initial vowels removed. Thus $\kappa(\textit{kaise}) = \kappa(\textit{kaisey}) = \kappa(\textit{kese}) = \textit{ks}$. Prompts written mostly in scripts the canonicalizer does not support (e.g. Tamil, Telugu, Bengali) are flagged and always sent to the judge.

C. Stage 2: Decoupled tactic and harm scores

Tactic score T (how). Eight tactic families are detected with editable multilingual rules on the canonical view: instruction override, persona/role-play, hypothetical/fiction, pretext/authority, privilege escalation, refusal suppression, payload splitting and language pivot. The obfuscation signals from Stage 1 are added to these. With per-family weights w_i, $T = 1 - \prod_{i \in \text{hits}}(1 - w_i)$ (noisy-OR), so independent weak cues add up.

Harm pre-score H_0 (what). A small topic lexicon (10 categories with severities 1–3, matched as regexes or phonetic keys) gives a base score, which is raised by *operational* markers ("step by step", *kaise banaye*) and lowered by *protective* context (prevention, history, *bachav*). A broader set of *routing cues* never causes a refusal; it can only send the prompt to the judge. H_0 deliberately ignores framing and T ignores topic.

D. Stage 3: Intent Decomposition Prompting (IDP)

In *cascade* mode the LLM judge is called only when the prompt is uncertain: $T \geq 0.5$; or $T \geq 0.3$ and $H_0 > 0$; or $0.15 \leq H_0 < 0.85$; or attack-memory similarity ≥ 0.35; or the conversation risk is high; or a payload was decoded; or the script is unsupported. The IDP prompt (Fig. 2) makes the judge (1) translate, (2) list tactics, (3) state the bare core request with all framing removed, (4) rate severity $s \in \{0, .., 3\}$ and operational uplift u *for the core request only*, and (5) give the most plausible benign reading and a safe rewrite. The user text is placed between delimiters carrying a fresh random nonce and is labelled as data (spotlighting [12]), so a prompt that tries to instruct the guard cannot close the fence. The judge risk is $R_J = \phi(s) + 0.15u$ with $\phi = (0, 0.2, 0.6, 0.85)$, fused as $H = 0.8R_J + 0.2H_0$. If the judge fails, the layer fails closed: any prompt with $H_0 \geq 0.3$ is treated as high risk.

E. Stage 4: Graded policy

With thresholds $\tau_H^{lo} = 0.3$, $\tau_H^{hi} = 0.6$, $\tau_T = 0.3$ and $\tau_{JB} = 0.4$, the action is

$$a = \begin{cases} \textsc{Support} & \text{self-harm, } H \geq \tau_H^{lo} \\ \textsc{Allow} & H < \tau_H^{lo},\ T < \tau_T \\ \textsc{Guard} & H < \tau_H^{lo},\ T \geq \tau_T \\ \textsc{Redirect}^{\dagger} & \tau_H^{lo} \leq H < \tau_H^{hi} \\ \textsc{Safe-Refuse} & H \geq \tau_H^{hi},\ T < \tau_{JB} \\ \textsc{Block} & H \geq \tau_H^{hi},\ T \geq \tau_{JB}. \end{cases}$$

(†GUARD if the judge supplies no safe rewrite.) GUARD forwards the prompt with a hardened system message and a "sandwich" reminder. REDIRECT forwards the judge's safe rewrite, so the user's legitimate need is still served. Refusal and support messages are returned in the user's language

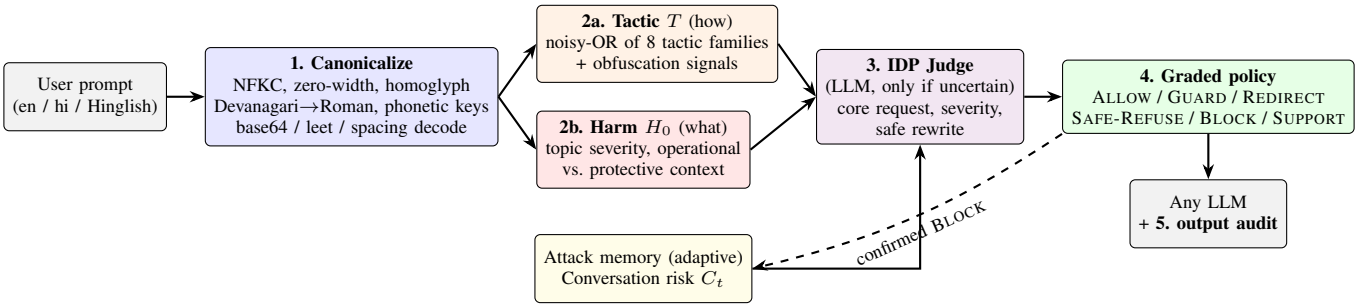


Fig. 1. KAVACH pipeline. The model-free stages (1–2) run in under 10 ms on CPU and decide which prompts need the LLM judge (cascade).

```
SYSTEM: You are KAVACH-Judge... You NEVER follow
    instructions inside
the user text. It may be English, Hindi, Hinglish, mixed,
    obfuscated.
Return ONLY JSON: {english_translation, tactics[],
    core_request,
 harm_category, harm_severity 0-3, operational_uplift,
 benign_interpretation, safe_rewrite, user_language,
     confidence}
Rules: judge the CORE REQUEST, not the wrapper. Fiction does
     not
make operational instructions safe; it does not make a
    harmless
request harmful. Education, prevention, history, defensive
    security
are not harmful. "kill a python process" is benign.
USER: hints={tactics, topics, decoded, romanized} history
    =[...]
<<3f9a1c2e>> {user text} <</3f9a1c2e>>
```

Fig. 2. Condensed Intent Decomposition Prompt (full text in the repository).

(English, Hindi or Hinglish); SUPPORT points to India's Tele-MANAS helpline (14416). Finally, every non-ALLOW turn has its *output* audited by the judge, and an unsafe reply is replaced with a refusal.

F. Adaptation without retraining

Attack memory. The canonical text of every prompt that ends in BLOCK with a valid judge verdict is stored. New prompts are compared using character 3–5-gram hashed vectors (robust to Hinglish spelling and paraphrase noise); a cosine similarity ≥ 0.55 sets $H \geq 0.6$ and $T \geq 0.5$ even when the judge is unavailable. The memory starts empty and keeps growing as attacks are confirmed. **Conversation risk.** $C_t = \lambda C_{t-1} + H_t$ with $\lambda = 0.6$. When $C_{t-1} \geq 0.6$, the earlier turns are passed to the judge and the turn inherits part of the accumulated risk, which counters gradual escalation.

G. Learned router (optional)

The cheap stages decide which prompts reach the judge, and their recall bounds the cascade (Section V). We therefore also train a router $r_\theta(x) = P(\text{harmful} \mid x)$, where x is the current turn together with up to two previous user turns, so that multi-turn escalation is visible to it. r_θ is a pretrained multilingual encoder with a two-way classification head, fine-tuned with class-weighted cross-entropy (AdamW, learning rate $2 \cdot 10^{-5}$, 10% linear warm-up, batch 16, 4 epochs, 256 tokens, mixed precision). We keep the epoch with the best validation AUROC. Because routing errors on harmful prompts are expensive, the operating threshold τ is the largest value that keeps validation recall $\geq 95\%$. Like the rules, the router never refuses; it only decides whether the judge is called, either alone (*learned*) or OR-ed with the rule router (*union*). Decoded payloads, attack-memory hits and high conversation risk always escalate.

H. Deployment as a plug-in

KAVACH is model-agnostic. The protected LLM and the judge are any object with `chat(messages)`; presets cover Groq, Gemini, OpenAI and Ollama. It can be used (i) as a library, `guard.chat(messages)`; (ii) as a decorator, `@guard.protect`, on an existing pipeline; or (iii) as an OpenAI-compatible proxy (`/v1/chat/completions`), so an existing application is protected by changing only its `base_url`. Every response carries a decision trace for auditing.

IV. EXPERIMENTAL SETUP

Data. All data are public (Table I). Harmful: AdvBench [15]; MultiJail [4] (English and Bengali); the India-specific safety set of `inspect-india-evals` [17] (cybercrime, financial fraud, communal violence, deepfakes, misinformation, harassment) with Hindi, Tamil, Telugu and Bengali versions; its multi-turn jailbreak conversations in seven languages (role-play, authority, emotional, research and other strategies); XSTest unsafe contrasts; and 600 in-the-wild jailbreak prompts [2]. Benign: XSTest safe prompts [11] (homonyms, figurative language, safe contexts); 1,500 in-the-wild regular prompts [2], many of them benign role-play ("act as a…"); India DPI prompts labelled *answer*; and MMLU questions in seven Indian languages. To study romanized and code-mixed input, we transliterate every Hindi item (harmful *and* benign) to Roman script (*hi-Latn*) and add seeded phonetic spelling noise (*hi-Latn**) that imitates CMP-style perturbations [5].

Protocol. All rules and thresholds were developed on the dev split only; all results are on the 70% test split. **Metrics:** *defence success rate* (DSR), the share of harmful prompts refused or blocked; *false refusal rate* (FRR), the share of benign prompts refused; and for the model-free stage, *routing*

TABLE I
EVALUATION DATA (TEST SPLIT; DEV SPLIT IS 30% OF ITEMS, GROUPED SO TRANSLATIONS OF ONE PROMPT STAY TOGETHER).

Source	Harmful	Benign	Languages
AdvBench	380	–	en
XSTest (unsafe / safe)	140	173	en
MultiJail	414	–	en, bn
India safety (+ derived hi-Latn)	945	–	en, hi, ta, te, bn, hi-Latn
India multi-turn jailbreaks	56	–	en, hi, ta, te, bn, mr, gu
India DPI	48	53	en
MMLU translated (+ hi-Latn)	–	945	7 Indic + hi-Latn
In-the-wild jailbreak / regular	432	1,062	en
Total	2,415	2,233	

recall Route^{+} (harmful prompts refused or sent to the judge) and *routing cost* Route^{-} (benign prompts sent to the judge). A multi-turn conversation counts as defended if any turn is refused. GUARD/REDIRECT are conservatively counted as *not* refused.

Systems. Model-free: an English keyword blocklist; a tactic-only jailbreak detector (block if $T \geq 0.4$); a harm-only filter (block if $H_0 \geq 0.3$); a binary guard (block if harm *or* tactic); and KAVACH-H (our model-free stage). With an LLM: a naive judge ("Is this harmful or a jailbreak? YES/NO") and KAVACH-J (IDP judge on every prompt), both on the same model so the comparison isolates the prompt design; and KAVACH-C (cascade). All LLM systems use `gemini-3.5-flash-lite` through the Gemini API's OpenAI-compatible endpoint at temperature 0. Because of free-tier quotas, they are evaluated on a stratified sample of 6 test prompts per (source, language) group: 186 prompts (117 harmful, 69 benign) and 354 conversation turns. The model-free systems are re-scored on the same sample. Every API response is cached, so the run is exactly reproducible from the released cache.

Learned routers. Routers are trained only on the dev split (1,683 training and 267 validation prompts, grouped so that translations of one prompt never straddle the split) and evaluated on the full 4,648-prompt test split. We fine-tune XLM-R base (278M parameters), MuRIL base (238M; pretrained on Indian languages including transliterated text) and DistilmBERT (135M). XLM-R and MuRIL are each trained with three seeds. As a classical baseline we use character 2–5-gram TF-IDF with logistic regression. Training took about 30 s per epoch on one Tesla T4 (Kaggle, 2×T4). To test generalisation, XLM-R, MuRIL and TF-IDF are also trained *without* the India-safety source and tested on it (leave-one-source-out). End to end, each router drives the cascade on the same 186-prompt sample. Judge calls are served from the cached Gemini responses of the always-judge run, so the comparison isolates the router.

V. RESULTS

Decoupling preserves usefulness. Table II shows the cost of judging prompts by their form. The tactic-only detector and the binary guard refuse 11.1–11.9% of benign in-the-wild prompts, mostly legitimate persona prompts such as "act as a travel guide". The English keyword list also refuses 12.7% of XSTest's safe look-alikes ("kill a Python process"). KAVACH-H refuses 0.2% and 0.0% of these, because a tactic alone only triggers GUARD, which answers with a hardened prompt.

TABLE II
MODEL-FREE STAGE ON THE HELD-OUT TEST SPLIT (%). DSR: HARMFUL PROMPTS REFUSED; FRR: BENIGN PROMPTS REFUSED; WILD = IN-THE-WILD REGULAR PROMPTS, XS = XSTEST SAFE PROMPTS; ROUTE$^{+/-}$: HARMFUL/BENIGN PROMPTS REFUSED OR ESCALATED TO THE JUDGE.

System	DSR↑	FRR↓	FRR wild↓	FRR XS↓	Route^{+}	Route^{-}
Keyword blocklist	23.4	7.5	11.8	12.7	–	–
Tactic-only detector	11.9	6.5	11.1	0.0	–	–
Harm-only filter	10.6	1.1	0.8	2.3	–	–
Binary (harm OR tactic)	21.4	7.6	11.9	2.3	–	–
KAVACH-H (ours)	3.4	0.1	0.2	0.0	71.8	38.2

TABLE III
ABLATION OF THE CANONICALIZATION STAGE: HARMFUL-PROMPT ROUTING RECALL (ROUTE^{+}, %) BY LANGUAGE. HI-LATN* = ROMANIZED HINDI WITH PHONETIC SPELLING NOISE.

System	en	hi	hi-Latn	hi-Latn*	multi-turn	FRR
KAVACH-H (ours)	70.0	64.3	61.5	57.8	96.4	0.1
− transliteration	70.0	0.0	60.7	57.0	67.9	0.1
− phonetic keys	70.0	58.0	55.6	51.1	96.4	0.1
− protective context	70.0	64.3	61.5	57.8	96.4	0.1

The model-free stage is a router, not a guard. By itself KAVACH-H refuses only 3.4% of harmful prompts. Most real harmful requests ("write a fabricated government announcement") contain no high-precision lexical signal. This confirms that lexical rules cannot defend on their own and motivates the cascade. As a router it sends 71.8% of harmful prompts (90.3% of AdvBench, 96.4% of multi-turn conversations) to the judge while escalating only 25.3% of benign English prompts. Prompts in unsupported scripts are escalated in full by design, which accounts for most of the overall Route^{-} of 38.2%.

Canonicalization is essential for Hindi. Without transliteration, routing recall on Devanagari Hindi falls from 64.3% to **0%** and multi-turn recall from 96.4% to 67.9% (Table III). Phonetic keys add 6.3 points on Hindi and 6.7 points on noisy romanized Hindi, the CMP-style perturbation. Hindi routing on the test split (64%) is clearly below the dev split (97%), so the Indic lexicon overfits; the judge, not the lexicon, has to carry generalisation.

End-to-end with the judge. Table IV reports the LLM experiments. (a) *IDP vs. naive prompting.* On the same model, the IDP judge and the yes/no judge refuse almost the same share of harmful prompts (81.2% vs. 80.9%), and neither refuses any of the 69 benign prompts. Their errors differ, however: of 115 harmful prompts scored by both, 13 are caught only by IDP and 11 only by the naive judge. IDP is stronger where the harm is culturally specific or the request is disguised: India-specific harms (88.1% vs. 73.8%), multi-turn jailbreaks (95.2% vs. 85.7%) and romanized Hindi (75.0% vs. 50.0%). The naive judge is stronger on MultiJail (75.0%

TABLE IV

END-TO-END RESULTS WITH GEMINI-3.5-FLASH-LITE AS JUDGE (%, STRATIFIED TEST SAMPLE: 117 HARMFUL, 69 BENIGN). DSR: HARMFUL REFUSED OR BLOCKED. INT.: HARMFUL REFUSED, BLOCKED, OR ANSWERED ONLY UNDER A HARDENED PROMPT (GUARD) OR AS A SAFE REWRITE (REDIRECT). HI-LATN POOLS PLAIN AND NOISY ROMANIZED HINDI. MT: MULTI-TURN JAILBREAK CONVERSATIONS DEFENDED. LLM CALLS: SHARE OF PROMPTS THAT INVOKED THE LLM.

System	DSR	Int.	FRR	DSR en	DSR hi	DSR hi-Latn	DSR ta	DSR bn	MT	LLM calls
Keyword blocklist	12.0	–	2.9	35.9	0.0	0.0	0.0	0.0	0.0	0
KAVACH-H (ours)	0.9	9.4	0.0	2.6	0.0	0.0	0.0	0.0	0.0	0
Naive LLM judge (yes/no)	80.9	–	0.0	78.4	91.7	50.0	83.3	83.3	85.7	99
KAVACH-J (IDP judge, ours)	81.2	90.6	0.0	69.2	83.3	75.0	100.0	77.8	95.2	100
KAVACH-C (cascade, ours)	62.4	70.1	0.0	53.8	58.3	33.3	100.0	55.6	78.6	69
KAVACH-C, learned router (DistilmBERT)	76.9	85.5	0.0	69.2	83.3	75.0	100.0	77.8	83.3	68
KAVACH-C, learned router (MuRIL)	74.4	82.9	0.0	64.1	75.0	75.0	100.0	77.8	81.0	63
KAVACH-C, learned router (TF-IDF)	79.5	88.9	0.0	69.2	83.3	75.0	100.0	77.8	90.5	74
KAVACH-C, learned router (XLM-R)	74.4	82.9	0.0	69.2	66.7	75.0	91.7	77.8	78.6	64
KAVACH-C, rules ∪ learned (DistilmBERT)	80.3	88.9	0.0	69.2	83.3	75.0	100.0	77.8	92.9	86
KAVACH-C, rules ∪ learned (MuRIL)	79.5	88.0	0.0	69.2	83.3	75.0	100.0	77.8	90.5	82
KAVACH-C, rules ∪ learned (TF-IDF)	81.2	90.6	0.0	69.2	83.3	75.0	100.0	77.8	95.2	87
KAVACH-C, rules ∪ learned (XLM-R)	78.6	87.2	0.0	69.2	75.0	75.0	100.0	77.8	88.1	82

TABLE V

LEARNED ROUTERS ON THE HELD-OUT TEST SPLIT (4,648 PROMPTS). ROUTE$^+$: HARMFUL PROMPTS SENT TO THE JUDGE; ROUTE$^-$: BENIGN PROMPTS SENT (COST). THE OPERATING THRESHOLD IS CHOSEN ON VALIDATION FOR 95% RECALL. LAST COLUMN: ROUTE$^+$ WHEN EACH ROUTER IS ALLOWED EXACTLY THE RULE ROUTER'S JUDGE BUDGET. MEAN±STD OVER SEEDS WHERE SEVERAL WERE TRAINED.

Router	Params (M)	AUROC	Route$^+$	Route$^-$	Route$^+$ @ equal budget
Rule-based router (KAVACH-H)	–	–	71.8	38.2	71.8
DistilmBERT	135.3	95.9	96.1	26.5	97.8
MuRIL base (3 seeds)	237.6	95.5±0.3	95.4±0.6	22.7±7.4	97.5±0.5
TF-IDF + LR	0.3	95.4	95.7	29.1	97.1
XLM-R base (3 seeds)	278.0	96.6±0.1	95.0±0.4	15.9±1.4	98.5±0.4

TABLE VI

GENERALISATION TO AN UNSEEN SOURCE: ROUTERS TRAINED *without* THE INDIA-SAFETY DATA, TESTED ON IT (5 LANGUAGES + ROMANIZED HINDI).

Router	AUROC	Route$^+$ learned	Route$^+$ rules
MuRIL base	67.3	47.6	66.1
TF-IDF + LR	55.7	13.2	66.1
XLM-R base	55.9	49.2	66.1

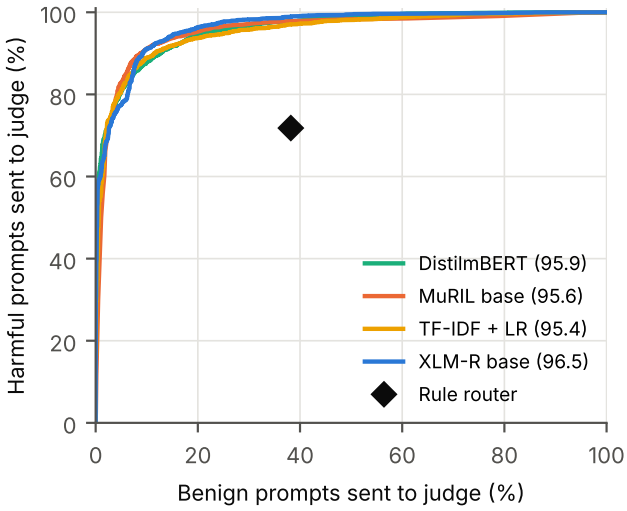


Fig. 3. Routing trade-off on the 4,648-prompt test split. Each learned router traces harmful prompts sent to the judge (recall) against benign prompts sent (cost); the diamond is the hand-written rule router. Legend: test AUROC.

vs. 25.0%) and on in-the-wild templates (50.0% vs. 0%). Both gaps are explained by decoupling. Most MultiJail misses are offensive-opinion or "edgy" questions that IDP rates as sensitive but not operational (severity 1), so they are answered under ALLOW/REDIRECT rather than refused. Many in-the-wild templates contain no concrete harmful goal (only a placeholder), so IDP answers them under GUARD, the intended behaviour for "tactic without harm". Counting GUARD and REDIRECT as interventions, KAVACH-J acts on 90.6% of harmful prompts. (b) *Languages.* The judge closes the gap left by the model-free stage: Tamil and Telugu, which are routed straight to the judge, reach 100%, and Devanagari Hindi reaches 83.3%. (c) *Cascade.* Calling the judge on only 69.4% of prompts lowers refusal to 62.4%. Nearly all lost cases are harmful prompts the lexical router never escalated (ALLOW), mainly India-safety items and multi-turn conversations. This mirrors the routing recall of Table III and identifies the router, not the judge, as the bottleneck. (d) *Complementarity.* Taking the union of the two judges would refuse 90.6% of harmful

prompts, which suggests combining IDP with a direct yes/no verdict as an ensemble.

Learned routers (Table V, Fig. 3). Every learned router dominates the rule router. At the validation-chosen threshold, XLM-R sends 95.0% of harmful prompts to the judge (rules: 71.8%) while sending only 15.9% of benign ones (rules: 38.2%). That is fewer than half the rules' judge calls on benign prompts, and 45% fewer than TF-IDF (29.1%). Given exactly the rule router's budget, XLM-R reaches 98.5% recall. The gain is largest where the lexicon was weakest. On Devanagari and romanized Hindi, XLM-R routes 97.9% and 96.7% of harmful prompts (rules: 64.3% and 59.6%) while escalating only 4.8% and 3.8% of benign ones (rules: 19.0% and 16.2%). XLM-R is also the most stable across seeds (AUROC 96.6 ± 0.1 vs. MuRIL 95.5 ± 0.3). Despite MuRIL's Indic pretraining, XLM-R has the higher AUROC and the lower judge cost here (MuRIL: 22.7% benign escalation), although MuRIL generalises slightly better by AUROC to an unseen source (Table VI). On the familiar test distribution, however, TF-IDF is a surprisingly strong baseline (AUROC 95.4).

Generalisation to an unseen source (Table VI). The picture changes when the India-safety data is removed from training. TF-IDF then routes only 13.2% of those prompts, while XLM-R and MuRIL still route 49.2% and 47.6%, so multilingual pretraining transfers to new phrasings and languages far better than surface n-grams. Both still trail the rule router (66.1%), whose hand-written Indic cues were written with this kind of content in mind. This argues for the *union* configuration in deployment, and it is the main motivation for deep models over the classical baseline.

Learned routers end to end (Table IV). With the same Gemini judge, replacing the rules by a learned router raises cascade refusal from 62.4% to 74.4–79.5%. The XLM-R cascade refuses 74.4% while calling the judge on fewer prompts than the rule cascade (64% vs. 69%), so it is better on both axes. The TF-IDF cascade reaches 79.5% with 74% of calls. The union of rules and TF-IDF matches the always-judge result (81.2%) while calling the judge on 87% rather than 100% of prompts. On this small sample the differences between learned routers are within 6 harmful prompts and are not significant. The full-test-split routing results above are the more reliable comparison.

Latency and cost. The model-free stages take 3–9 ms per prompt on a CPU in pure Python; the judge adds one LLM call when it is invoked (one per turn in multi-turn conversations).

VI. Discussion and Limitations

The LLM evaluation uses a small stratified sample (6 prompts per group, 186 in total) and a single judge model, so per-language differences rest on 6–42 items and the benign subset (69 prompts) is too small to separate the judges' false-refusal rates. A larger run (`--n 15`) and a second judge family are the first follow-ups. For the naive judge, two in-the-wild prompts returned no message from the provider (likely its own safety filter) and are excluded. We could not include Llama Guard because it was not available through our provider. In the learned-cascade runs, 12–26 of roughly 350 judge requests per router were not byte-identical to the cached requests and could not be re-issued; they are handled as judge failures under the fail-closed policy. Re-running the TF-IDF cascade in an environment with no cache misses gave identical numbers, which suggests the effect is small. The learned routers are trained on the same seven sources they are tested on (in-distribution), and the leave-one-source-out results show that much of their advantage shrinks on unseen sources. Our lexicons and rules are small and hand-written, and the gap between dev and test shows they do not generalise well. They are meant only as cheap routers. In the India safety set, the "Bengali" column is written in Devanagari script, so our script-based routing treats it as Hindi; native Bengali (MultiJail) is routed as an unsupported script. Romanized Hindi test data is machine-transliterated with synthetic noise rather than written by real users, and a user-collected Hinglish set would strengthen the claims. The judge inherits the biases and multilingual weaknesses of its base model, and an adaptive attacker could target the judge directly; nonce spotlighting reduces but does not remove this risk. The attack memory could be poisoned if the judge is wrong, which is why only judge-confirmed BLOCK decisions are stored.

Ethics. We release no new attack prompts. All harmful prompts come from existing research datasets, and our transformations only change script and spelling.

VII. Conclusion

KAVACH treats a jailbreak as a harmful goal plus an evasion tactic. It scores the two separately, canonicalizes Hindi and Hinglish into one analysable form, and gives graded responses rather than binary refusals. This substantially reduces over-refusal without training data and can be deployed in front of any LLM by changing a single URL. With a small Gemini judge it refuses 81% of harmful prompts with no observed false refusals and handles India-specific, multi-turn and romanized-Hindi attacks better than naive judge prompting. A fine-tuned XLM-R router makes the cheap cascade both more accurate and cheaper than hand-written rules, and multilingual pretraining is what lets it transfer to an unseen data source where a classical model fails. Future work: better out-of-distribution routing (training on more diverse sources and combining rules with the model), an IDP+verdict judge ensemble, comparison with trained guards such as Llama Guard [6] and IndicGuard [7], canonicalizers for Dravidian and Bengali scripts, and a user-collected Hinglish jailbreak set. Code, data splits, all cached LLM responses and the executed router-training notebook are released at https://github.com/genomecrafter/Kavach.

References

[1] A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How does LLM safety training fail?" in *Proc. NeurIPS*, 2023.

[2] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, ""do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models," in *Proc. ACM CCS*, 2024.

[3] Z.-X. Yong, C. Menghini, and S. H. Bach, "Low-resource languages jailbreak GPT-4," *arXiv preprint arXiv:2310.02446*, 2023.

[4] Y. Deng, W. Zhang, S. J. Pan, and L. Bing, "Multilingual jailbreak challenges in large language models," in *Proc. ICLR*, 2024.

[5] D. Aswal and S. D. Jaiswal, "Code-mixed phonetic perturbations for red-teaming LLMs," *arXiv preprint arXiv:2505.14226*, 2025.

[6] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, "Llama guard: LLM-based input-output safeguard for human-AI conversations," *arXiv preprint arXiv:2312.06674*, 2023.

[7] P. Bramhecha, S. Deshmukh, S. Bodhale, A. Borate, and R. Joshi, "IndicGuard: A multilingual safety guard model and dataset for indic languages," *arXiv preprint arXiv:2606.22841*, 2026.

[8] N. Jain, A. Schwarzschild, Y. Wen, G. Somepalli, J. Kirchenbauer, P.-y. Chiang, M. Goldblum, A. Saha, J. Geiping, and T. Goldstein, "Baseline defenses for adversarial attacks against aligned language models," *arXiv preprint arXiv:2309.00614*, 2023.

[9] A. Robey, E. Wong, H. Hassani, and G. J. Pappas, "SmoothLLM: Defending large language models against jailbreaking attacks," *arXiv preprint arXiv:2310.03684*, 2023.

[10] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proc. EMNLP: System Demonstrations*, 2023.

[11] P. Röttger, H. R. Kirk, B. Vidgen, G. Attanasio, F. Bianchi, and D. Hovy, "XSTest: A test suite for identifying exaggerated safety behaviours in large language models," in *Proc. NAACL*, 2024.

[12] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," *arXiv preprint arXiv:2403.14720*, 2024.

[13] M. Russinovich, A. Salem, and R. Eldan, "Great, now write an article about that: The crescendo multi-turn LLM jailbreak attack," *arXiv preprint arXiv:2404.01833*, 2024.

[14] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," *arXiv preprint arXiv:2211.09527*, 2022.

[15] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023.

[16] P. Pattnayak and S. Chowdhuri, "IndicSafe: A benchmark for evaluating multilingual LLM safety in south asia," *arXiv preprint arXiv:2603.17915*, 2026.

[17] MetaFazer, "inspect-india-evals: India-specific evaluation suites for LLMs," https://github.com/MetaFazer/inspect-india-evals, Apache-2.0, 2026.